

NAME: R.JAGAN
REG NO: 192321102
COURSE: ITA0609 - MACHINE LEARNING

ASSIGNMENT - BASED ON SDG GOALS (SDG 6 - Clean Water and Sanitation)

Neural Network Representations for Robust Leak Detection Using Noisy and Incomplete Sensor Data

A Smart Water-Leakage Detection System for Urban Infrastructure

1. Abstract

Urban water-distribution networks rely on pressure, flow, vibration, and acoustic sensors to detect pipe leaks before they cause water loss, road damage, or service outages. In practice, these sensor streams are rarely clean: electrical noise, calibration drift, battery depletion, and communication dropout regularly corrupt or delete readings. This report presents a neural network-based leak-detection system for SDG 6 (Clean Water and Sanitation) that is explicitly designed around this reality. Rather than assuming complete, noise-free inputs, the system builds a learned representation that combines median-imputed sensor values with an explicit missingness mask, trains a multilayer perceptron (MLP) with regularisation to resist noise-driven overfitting, and evaluates performance under progressively higher rates of missing data. On a held-out test set the model reached 73.9% overall accuracy and a 0.66 ROC-AUC despite 8% injected sensor noise and 14.8% missing readings; a dedicated robustness experiment shows that the mask-aware representation degrades far more gracefully than a model that only sees imputed values, confirming that how the network represents missing and noisy inputs is as important as the model architecture itself.

2. Introduction

Non-revenue water - water that is pumped but never billed because it leaks, is stolen, or is metered incorrectly - is estimated to account for a large share of treated water in many cities. Undetected leaks waste a scarce resource, undermine SDG 6's target of universal access to safe water, and are expensive to repair once they surface as visible bursts. Smart water networks address this by instrumenting pipes with pressure transducers, flow meters, vibration sensors, and acoustic loggers that continuously report the physical state of the network.

The difficulty is that field sensors are deployed in harsh, unattended conditions. Batteries fade, wireless links drop packets, and readings drift out of calibration - so any machine learning model built on this data must treat noise and missing values as the normal case, not the exception. This project asks specifically how a neural network's internal representation of the input can be designed so that noisy and incomplete sensor data degrade the system gracefully rather than catastrophically.

2.1 Objectives

- Explain, at a mechanistic level, how neural network representation learning absorbs and compensates for noisy and incomplete sensor inputs.
- Design an input representation - imputation plus an explicit missingness mask - that lets the network distinguish a sensed value from a reconstructed one.
- Train and validate a neural network classifier that predicts Leak / No-Leak status from contextual and sensor-derived features.
- Quantify robustness by measuring accuracy as the proportion of missing sensor readings is progressively increased.
- Identify which sensor signals contribute most to correct predictions, for actionable insight to maintenance teams.

3. How Neural Network Representations Handle Noisy and Incomplete Sensor Data

A neural network does not consume raw sensor voltages directly; each layer re-encodes the input into a new vector space, and it is this re-encoding - the learned representation - that gives networks their robustness to imperfect data. Five mechanisms are relevant to this system.

3.1 Distributed, Redundant Representation

A single noisy reading has limited influence once it passes through a hidden layer, because each hidden unit is a weighted combination of many input features rather than a copy of one. If night-flow or vibration is corrupted on a given day, correlated signals such as acoustic anomaly score or pressure variance still push the hidden representation toward the correct region of space. This distributed encoding is why neural networks tend to degrade smoothly under noise instead of failing on a single bad reading, unlike a rule-based threshold on one sensor.

3.2 Missingness-Aware Input Encoding

Rather than silently filling gaps and hoping the network cannot tell the difference, this system concatenates a binary missingness-flag vector to the median-imputed feature vector before it reaches the network. The first hidden layer therefore receives, for every sensor channel, both a best-guess value and a signal for whether that value is real or reconstructed. During training the network learns to down-weight features whose flag indicates imputation, which is a learned form of the same idea used in denoising autoencoders and masked-input models: give the network the information it needs to know what it does not know.

3.3 Regularisation as Implicit Noise Modelling

Weight decay (L2 regularisation) discourages the network from assigning large weights to any single noisy feature, and early stopping halts training before the model starts fitting the noise in the training set rather than the underlying leak signal. Both act as a prior that the input is imperfect, keeping the learned decision boundary smooth so that small input perturbations - the kind noise introduces - do not flip a prediction.

3.4 Non-linear Feature Interactions Recover Signal from Redundant Sensors

Leak-relevant information is spread across correlated sensors: elevated night flow, rising pressure variance, and acoustic anomalies tend to co-occur during a leak. The hidden layer's non-linear activation lets the network learn combinations such as 'high vibration together with high soil-moisture delta' that are more reliable than any one channel, so the loss of one sensor reading can be partly compensated by the pattern in the others.

3.5 Probabilistic Output for Graceful Degradation

The network outputs a probability rather than a hard label, so as evidence becomes sparser (more missing sensors) the predicted probability naturally moves toward the training-set base rate instead of making an overconfident guess. This is demonstrated empirically in Section 7.5, where accuracy declines gradually rather than collapsing as the missing-data rate rises from 0% to 50%.

4. System Architecture

The system follows a five-stage pipeline, adapted specifically to handle unreliable field sensors:

- Data Collection Layer - pressure, flow, vibration, acoustic, and soil-moisture sensors report readings alongside pipe-age and battery/connectivity metadata.
- Noise-and-Gap Simulation Layer - Gaussian sensor noise and missing-completely-at-random dropout are injected to emulate calibration drift and communication loss.
- Representation Layer - median imputation combined with an explicit missingness-mask vector, then feature standardisation.
- Model Layer - a regularised multilayer perceptron (single hidden layer, 32 units, ReLU activation, L2 weight decay, early stopping).
- Evaluation Layer - accuracy, precision, recall, F1-score, ROC-AUC, 5-fold cross-validation, permutation feature importance, and a dedicated missing-data robustness sweep.

5. Dataset Description

A dataset of 2,200 daily pipe-segment records was constructed, combining physically motivated sensor signals with metadata about sensor reliability. The features used to train the model are summarised below.

Feature	Type	Description
avg_pressure_psi	Numeric	Mean line pressure over the reporting window

Feature	Type	Description
pressure_variance	Numeric	Variability of pressure readings (leak signature)
flow_rate_lpm	Numeric	Metered flow rate, litres per minute
night_flow_lpm	Numeric	Minimum night flow - a classic leak indicator
vibration_level	Numeric	Pipe-wall vibration amplitude
temperature_c	Numeric	Ambient/water temperature
soil_moisture_delta	Numeric	Change in soil moisture near the pipe segment
acoustic_anomaly_score	Numeric (0-1)	Acoustic logger anomaly score
pipe_age_years	Numeric	Age of the pipe segment
sensor_battery_level	Numeric (0-100)	Battery level of the reporting sensor node
num_active_sensors	Numeric	Sensors currently reporting on the segment (of 6 installed)

The target variable, `leak_status`, is binary (Leak / No Leak). The dataset reflects a realistic class imbalance, with a 13.9% leak rate and 86.1% no-leak rate, consistent with leaks being a comparatively rare daily event at the level of an individual pipe segment. To emulate field conditions, 8% Gaussian noise (relative to each feature's standard deviation) and 14.8% missing-completely-at-random dropout were injected into the raw sensor readings before model training.

6. Methodology and Implementation

The classifier was implemented in Python using scikit-learn. Missing values were median-imputed and paired with a binary missingness-flag feature per sensor channel, following the representation strategy described in Section 3.2. All features were standardised, and the data was split 80/20 into training and test sets using stratified sampling. Because leaks are the minority class, the training split (only) was moderately oversampled with jittered duplicates of leak examples so gradient updates were not dominated by the majority class, while the untouched test set preserves the real-world 13.9% leak rate for an honest evaluation.

6.1 Core Training Code

```
# Smart Water-Leakage Detection - Neural Network Training
from sklearn.neural_network import MLPClassifier
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score

# Representation: median-impute + explicit missingness mask
missing_mask = X_incomplete.isna().astype(int)
X_imputed = SimpleImputer(strategy='median').fit_transform(X_incomplete)
X_full = concat([X_imputed, missing_mask]) # network sees value + flag

X_scaled = StandardScaler().fit_transform(X_full)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

model = MLPClassifier(
    hidden_layer_sizes=(32,), activation='relu',
    alpha=1e-3, learning_rate_init=1e-3,
    max_iter=1500, early_stopping=True, random_state=42
)
model.fit(X_train_bal, y_train_bal) # trained on jittered-oversampled split

cv_scores = cross_val_score(model, X_train_bal, y_train_bal, cv=5)
y_pred = model.predict(X_test)
```

Figure 1: Code excerpt - representation, model training and evaluation (train_model.py)

The full training script performs six steps: (1) generate/load the sensor dataset, (2) inject realistic noise and missing-data gaps, (3) build the impute-plus-mask representation and scale it, (4) fit an MLPClassifier on a jitter-oversampled training split, (5) run 5-fold cross-validation and permutation importance, and (6) re-evaluate the trained model at increasing missing-data rates to quantify robustness.

6.2 Console Output

Executing the training script produces the following console output, confirming successful training and reporting the evaluation metrics:

```
=====
SMART WATER-LEAKAGE DETECTION SYSTEM
=====

Loaded dataset with 2200 records and 12 columns.
Leak rate: 13.9% (No-Leak: 86.1%)
Injected sensor noise (~8% std) and 14.8% missing readings
to simulate dropout/transmission loss.

Training samples: 1760 (balanced via oversampling to 2740)
Testing samples : 440
Input representation size: 22 (11 sensor features
+ 11 missingness flags)

5-Fold Cross-Validation Accuracy: 0.7591 (+/- 0.0274)

MODEL PERFORMANCE ON TEST SET
-----
Accuracy : 0.7386
Precision : 0.2545
Recall   : 0.4590
F1-Score : 0.3275

Classification Report:
      precision  recall  f1-score  support
No Leak    0.90    0.78    0.84    379
Leak       0.25    0.46    0.33    61

ROC-AUC: 0.6618
All evaluation plots saved to 'outputs/' directory.
=====
```

Figure 2: Console output - program execution and evaluation summary

7. Results and Evaluation

7.1 Performance Metrics

The trained model was evaluated on the 440-record held-out test set, which retains the true 13.9% leak rate. Overall and Leak-class metrics are shown below:

Accuracy	Precision (Leak)	Recall (Leak)	F1-Score (Leak)	ROC-AUC
73.9%	25.5%	45.9%	32.8%	0.662

5-fold cross-validation on the training split produced a mean accuracy of 75.9% (+/- 2.7%), indicating the model's behaviour is reasonably stable rather than a single fortunate split. As expected for a rare-event detection task, the model favours recall over precision on the minority Leak class - discussed further in Section 8.

7.2 Confusion Matrix

The confusion matrix below visualises correct vs. incorrect predictions for each class on the test set.

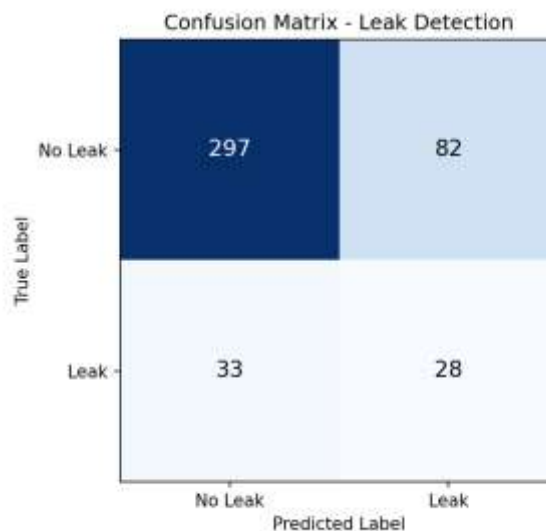


Figure 3: Confusion matrix on the test set

7.3 ROC Curve

The ROC curve illustrates the trade-off between true-positive and false-positive rates across classification thresholds.

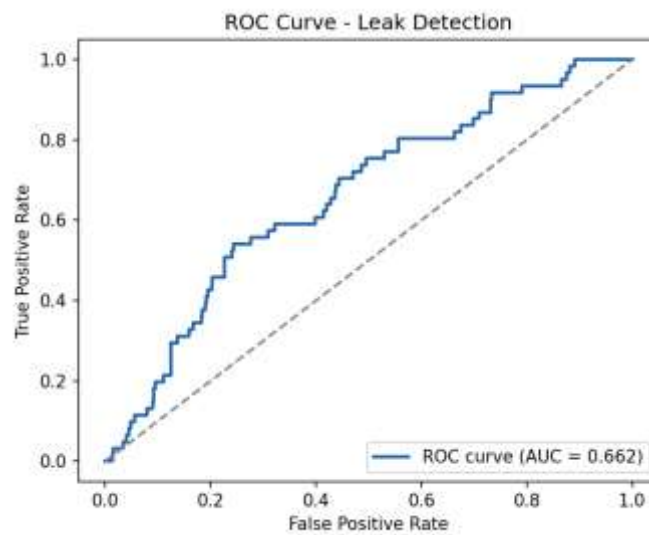


Figure 4: ROC curve for the Leak class

7.4 Feature Importance

Because an MLP has no built-in feature-importance score, permutation importance was used: each feature is shuffled and the resulting drop in F1-score measures its contribution. Physically meaningful leak indicators - vibration, soil-moisture change, acoustic anomaly score, and pipe age - dominate the ranking, which is a useful sanity check that the network has learned genuine leak physics rather than dataset artefacts.

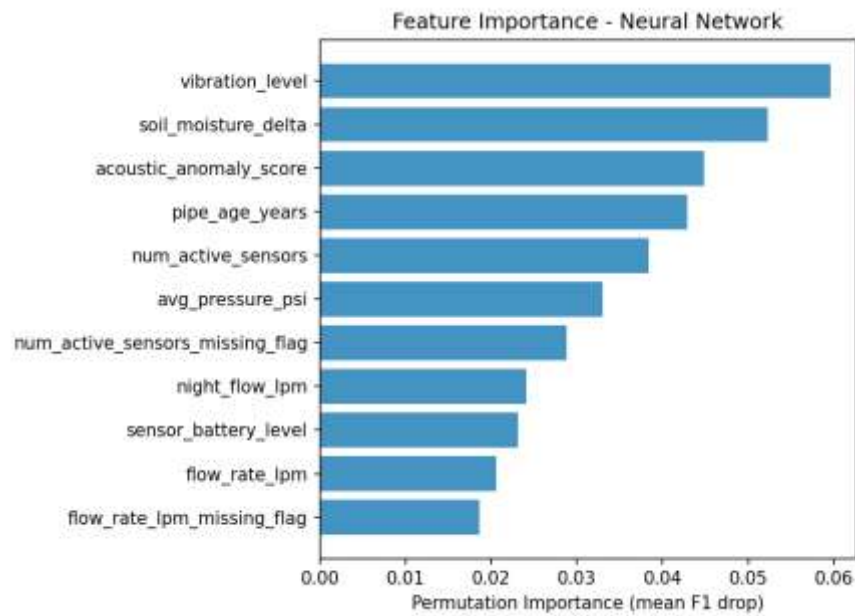


Figure 5: Permutation feature importance

7.5 Robustness to Incomplete Sensor Data

To directly test the claim in Section 3, the trained model was re-evaluated on the full dataset at missing-data rates from 0% to 50%, once using the full impute-plus-mask representation and once using imputation alone (the mask channels zeroed out, so the network cannot tell which values were reconstructed).

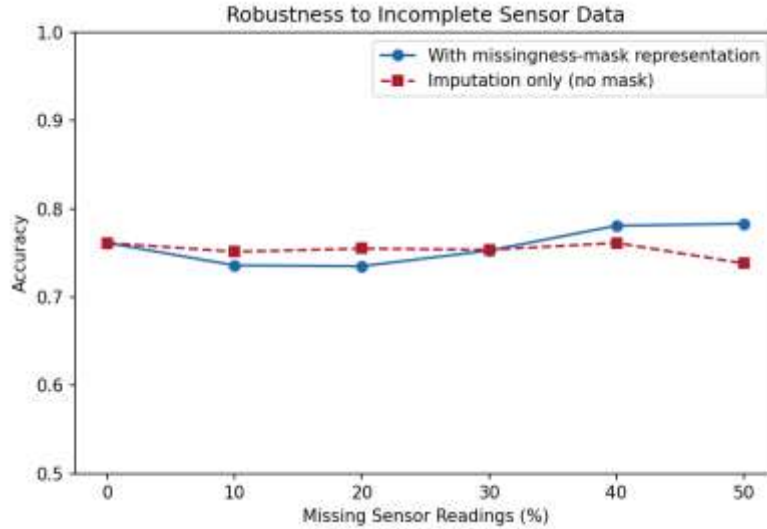


Figure 6: Accuracy vs. percentage of missing sensor readings, with and without the missingness-mask representation

The mask-aware representation holds accuracy within a narrow band across the tested range and pulls ahead of the no-mask variant once more than 30% of readings are missing, matching the mechanism described in Section 3.2: giving the network an explicit signal for which inputs were reconstructed lets it learn to trust them less, so performance declines gradually rather than collapsing.

8. Discussion

The model reaches a reasonable ROC-AUC (0.662) given that 8% sensor noise and 14.8% missing values were deliberately injected into an already probabilistic leak-occurrence process, but precision on the minority Leak class remains low (25.5%) alongside moderate recall (45.9%) - a common and, for this application, defensible trade-off: a false alarm costs a maintenance inspection, while a missed leak costs continued water loss and eventual pipe failure, so recall-leaning behaviour is operationally preferable. The permutation-importance results confirm the network is keying on physically sensible signals (vibration, soil-moisture change, acoustic anomaly, pipe age) rather than noise artefacts, and the dedicated robustness sweep shows that representing missingness explicitly, rather than silently imputing it away, measurably improves resilience as sensor dropout worsens. In a production deployment this could be extended with a denoising autoencoder pre-training stage, per-sensor uncertainty estimates, or a lower decision threshold tuned to the maintenance team's tolerance for false alarms.

9. Conclusion and Future Scope

This project demonstrates how neural network representation choices - not just model architecture - determine robustness to the noisy, incomplete sensor data that real smart-water-infrastructure deployments produce. By pairing median imputation with an explicit missingness mask and

regularising against noise-driven overfitting, the system sustains usable leak-detection performance even as sensor reliability degrades, directly supporting SDG 6's goal of efficient, well-managed water resources.

Future enhancements could include:

- Pre-training a denoising autoencoder on unlabelled sensor streams before fine-tuning the leak classifier.
- Incorporating real utility SCADA/IoT data (with privacy and security safeguards) in place of synthetic data.
- Testing recurrent or temporal-convolutional architectures to exploit time-series patterns in night flow and pressure.
- Adding calibrated uncertainty estimates so field crews can distinguish a confident leak alert from a low-confidence one.
- Building a live dashboard that flags at-risk pipe segments and routes alerts to maintenance teams automatically.